PES UNIVERSITY

SOFTWARE ENGINEERING LABORATORY

LAB 7

FUZZING FOR INPUT VALIDATION BUGS

Student Details	
Name	Deeraj S
SRN	PES2UG25CS806
Course	Software Engineering
Experiment	Lab 7 – Fuzzing for Input Validation Bugs
Date	11 November 2026

1. Objective

To explore fuzz testing as a security and quality assurance technique by identifying and fixing input-validation bugs in a Python program using the Hypothesis fuzzing library.

2. Software Requirements

- Python 3.8 or later
- Hypothesis
- pytest
- Python editor (Visual Studio Code)
- Git and GitHub

3. Learning Outcomes

- Understand the purpose and methodology of fuzz testing.
- Apply property-based fuzzing using Hypothesis.
- Detect and analyse bugs caused by edge-case inputs.
- Improve code robustness through defensive programming.

4. Introduction to Fuzz Testing

Fuzzing is a dynamic testing technique in which a program is supplied with random, unexpected, malformed, or unusual inputs to discover crashes, incorrect behaviour, and potential security weaknesses. It helps uncover edge cases that may be missed by manual testing.

Traditional unit tests check selected examples. Fuzz testing automatically explores a broader range of inputs, such as empty strings, unusual characters, malformed formats, and values of unexpected types.

5. Hypothesis Library

Hypothesis is a property-based testing library for Python. Tests use the @given decorator with strategies that describe the input values to generate. Hypothesis generates examples, checks the required property, and can minimise failing inputs to make debugging easier.

Strategy	Purpose
st.text()	Generates strings, including empty and unusual Unicode strings.
st.none()	Generates None values.
st.text() | st.none()	Generates either a string or None.

6. Application Under Test

The assigned application is an Income Tax Calculator parser. It processes PAN values, rupee amounts, and tax-slab strings through three functions.

Function	Purpose	Validation behaviour
clean_pan(pan)	Removes spaces and converts a PAN value to uppercase.	Returns an empty string for non-string input.

parse_amount(text)	Converts a formatted amount such as Rs 12,50,000 into an integer.	Returns None for invalid or non-numeric input.
parse_tax_slab(line)	Parses LOW-HIGH:RATE into a tuple.	Returns None for malformed or invalid ranges.

7. Procedure

1. **Install dependencies:** Installed Hypothesis and pytest using pip.
2. **Inspect and run the application:** Executed income_tax_parser.py with sample PAN, amount, and tax-slab values.
3. **Complete the fuzz tests:** Used Hypothesis strategies to generate strings and None values for all three functions.
4. **Run the tests:** Executed python -m pytest -v test_income_tax_parser.py.
5. **Fix the parser:** Added input-type checks, format validation, and safe handling for invalid values.
6. **Re-run the tests:** Executed the fuzz tests against the corrected parser.
7. **Version control:** Committed and pushed the completed LAB7 folder to the combined GitHub repository.

8. Implementation Details

8.1 clean_pan()

Checks whether the input is a string. Non-string input returns an empty string. Valid strings have spaces removed and are converted to uppercase.

8.2 parse_amount()

Checks the input type, trims whitespace, removes the Rs prefix, commas, and spaces, verifies that the remaining text contains only digits, and converts it to an integer. Invalid input returns None.

8.3 parse_tax_slab()

Checks the input type and parses the LOW-HIGH:RATE format. It converts the three fields to integers, rejects negative values, and checks that the lower limit does not exceed the upper limit. Malformed input returns None.

9. Test Execution and Results

Command used:

```
python -m pytest -v test_income_tax_parser.py
```

The final test run collected three Hypothesis tests. All three completed successfully.

Test case	Function tested	Result
test_clean_pan_no_crash	clean_pan()	PASSED
test_parse_amount_no_crash	parse_amount()	PASSED
test_parse_tax_slab_no_crash	parse_tax_slab()	PASSED

Final result: 3 passed.

10. Observations

- Hypothesis can explore inputs beyond a small set of manually selected examples.
- Type checks prevent string operations from being applied to incompatible values.
- Format and range validation help prevent malformed amounts and tax slabs from being accepted.

- The final test run completed successfully with all three no-crash tests passing.

11. Conclusion

Fuzz testing was applied to the Income Tax Calculator parser using Hypothesis. The three parser functions were improved with defensive input validation, and the final test execution reported three passing tests. The lab demonstrated how property-based testing can expose edge cases and improve software robustness.

12. GitHub Repository

Combined repository: https://github.com/DEERAJSCoding/PES2UG25CS806_SE_LAB

The completed Lab 7 files are stored in the LAB7 folder.

13. Source Code – income_tax_parser.py

Complete corrected application source code:

```
def clean_pan(pan):
    """
    Removes spaces from a PAN and converts it to uppercase.
    Returns an empty string for invalid input.
    """
    if not isinstance(pan, str):
        return ""

    return pan.strip().replace(" ", "").upper()



def parse_amount(text):
    """
    Converts an amount such as 'Rs 12,50,000' into an integer.
    Returns None for invalid input.
    """
    if not isinstance(text, str):
        return None

    text = text.strip()

    if not text:
        return None

    text = text.replace("Rs", "").replace("rs", "")
    text = text.replace(",", "").replace(" ", "").strip()

    if not text.isdigit():
        return None

    return int(text)



def parse_tax_slab(line):
    """
    Parses 'LOW-HIGH:RATE' into a (low, high, rate) tuple.
    Returns None for invalid input.
    """
    if not isinstance(line, str):
        return None

    try:
        limits, rate = line.split(":")
        low, high = limits.split("-")

        low = int(low.strip())
        high = int(high.strip())
        rate = int(rate.strip())

        if low < 0 or high < 0 or rate < 0:
            return None

        if low > high:
            return None

        return low, high, rate

    except (ValueError, AttributeError):
        return None



def main():
    print("==== Running clean_pan ====")
    value = input("Enter a PAN (e.g. abcde 1234f): ")
    print("Cleaned PAN:", clean_pan(value))
```

```
    print("\\n==== Running parse_amount ====")
    value = input("Enter an amount (e.g. Rs 12,50,000): ")
    result = parse_amount(value)
    print("Amount in rupees:", result)

    print("\\n==== Running parse_tax_slab ====")
    value = input("Enter a tax slab as LOW-HIGH:RATE (e.g. 250000-500000:5): ")
    result = parse_tax_slab(value)
    print("Tax slab (low, high, rate):", result)


if __name__ == "__main__":
    main()
```

14. Fuzz Test Code – test_income_tax_parser.py

Completed Hypothesis tests using try/except to verify that generated inputs do not cause unhandled crashes:

```
from hypothesis import given, strategies as st
from income_tax_parser import clean_pan, parse_amount, parse_tax_slab


@given(st.text() | st.none())
def test_clean_pan_no_crash(s):
    try:
        clean_pan(s)
    except Exception as e:
        assert False, f"clean_pan crashed with: {e}"

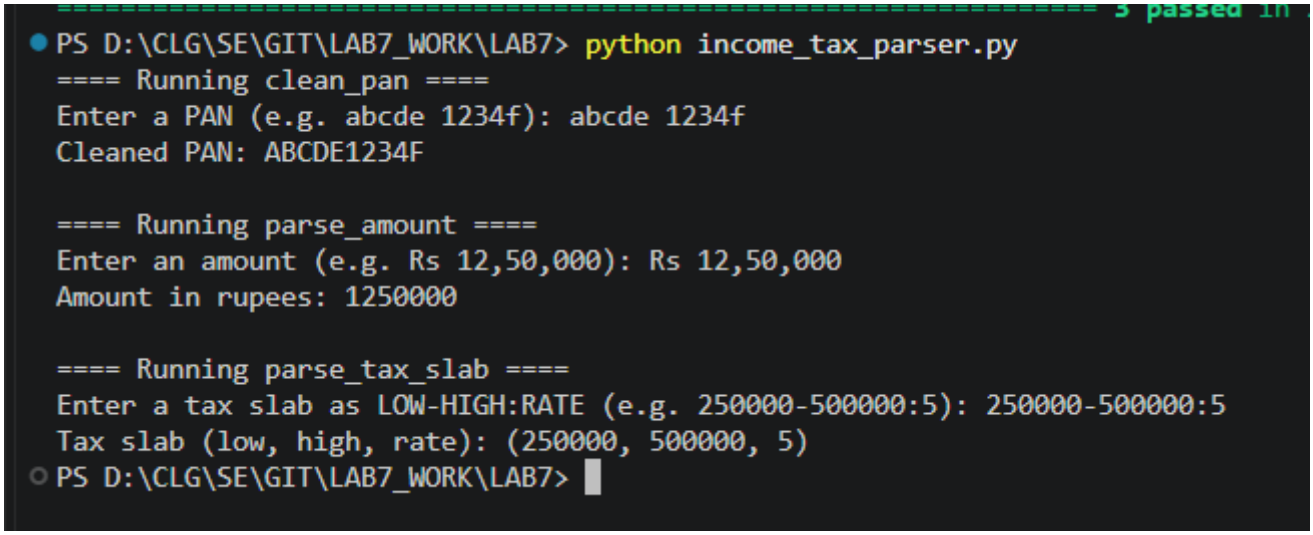

@given(st.text() | st.none())
def test_parse_amount_no_crash(s):
    try:
        parse_amount(s)
    except Exception as e:
        assert False, f"parse_amount crashed with: {e}"


@given(st.text() | st.none())
def test_parse_tax_slab_no_crash(s):
    try:
        parse_tax_slab(s)
    except Exception as e:
        assert False, f"parse_tax_slab crashed with: {e}"
```

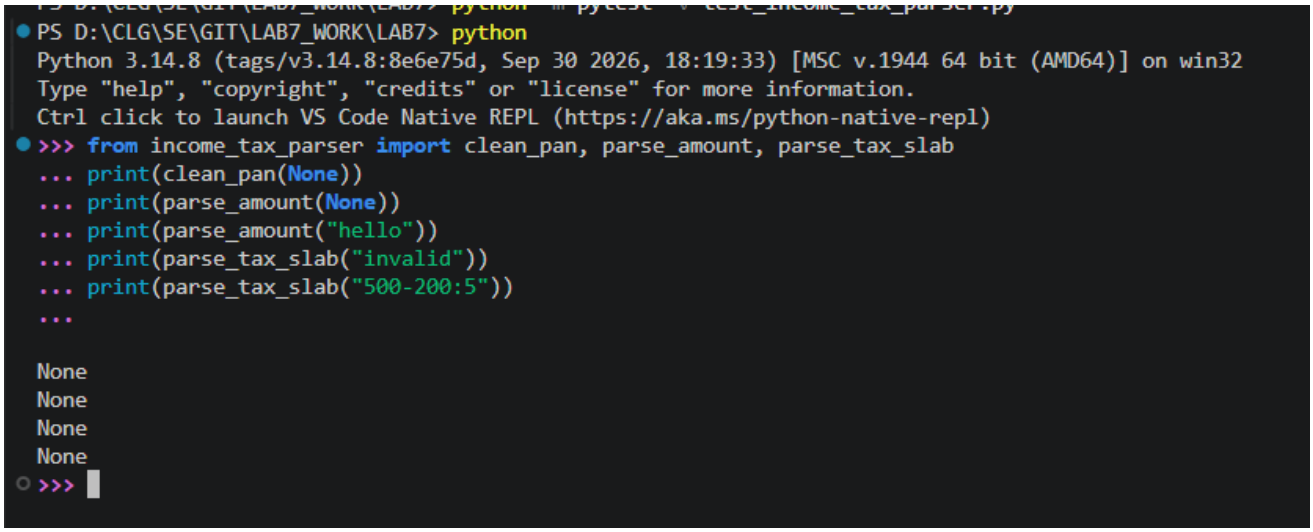
15. Screenshot Evidence

The following screenshots are taken from the lab work shared during this conversation. They document the application run, invalid-input handling, final test results, and GitHub submission.

15.1 Application execution with sample inputs



15.2 Invalid-input checks using the Python interpreter



15.3 Final Hypothesis test execution – all three tests passed

